

I.E.S. LA MARISMA — HUELVA
DEPARTAMENTO DE INFORMÁTICA
Ciclo Formativo de Grado Superior

DESARROLLO DE APLICACIONES MULTIPLATAFORMA
MÓDULO PROYECTO INTERMODULAR

SLIOR

Sistema de Optimización de Rutas Logísticas

Autor: José Ramos Contioso

Fecha: 10 de mayo de 2026

Tutora: María Arantzazu Fiallega Escauriaza

HOJA RESUMEN DEL PROYECTO

Título del proyecto:	SLIOR — Sistema de Optimización de Rutas Logísticas
Autor:	José Ramos Contioso
Fecha:	10 de mayo de 2026
Tutora:	María Arantzazu Fiallega Escauriaza
Titulación:	C.F.G.S. Desarrollo de Aplicaciones Multiplataforma
Centro:	I.E.S. La Marisma, Huelva
Palabras clave (ES):	logística, rutas, Android, Spring Boot, optimización, offline-first, Kotlin
Palabras clave (EN):	logistics, route optimization, Android, Spring Boot, offline-first, Kotlin

Resumen del proyecto (Español)

SLIOR es un sistema completo de optimización de rutas logísticas compuesto por una API REST desarrollada con Java y Spring Boot, y una aplicación móvil nativa Android desarrollada con Kotlin y Jetpack Compose. El sistema permite a los repartidores gestionar sus rutas de entrega de forma eficiente, con soporte para el modo sin conexión (offline-first) mediante una base de datos local Room, geocodificación de direcciones a través de Photon y Nominatim, y optimización automática del orden de visita mediante el algoritmo Nearest Neighbor. La aplicación incluye autenticación segura por JWT, escaneo de códigos de barras mediante ZXing, visualización de mapas con OSMDroid, y sincronización en segundo plano con WorkManager. Todo el sistema se diseña bajo una arquitectura MVVM con Clean Architecture y Repository Pattern, garantizando la separación de responsabilidades y la mantenibilidad del código.

Project Summary (English)

SLIOR is a complete logistics route optimization system consisting of a REST API built with Java and Spring Boot, and a native Android mobile application developed with Kotlin and Jetpack Compose. The system allows delivery drivers to efficiently manage their delivery routes, with support for offline-first mode through a local Room database, address geocoding via Photon and

Nominatim, and automatic visit order optimization using the Nearest Neighbor algorithm. The application includes secure JWT authentication, barcode scanning via ZXing, map visualization with OSMDroid, and background synchronization with WorkManager. The entire system is designed under an MVVM architecture with Clean Architecture and Repository Pattern, ensuring separation of concerns and code maintainability.

1. INTRODUCCIÓN

1.1 INTRODUCCIÓN A LA MEMORIA

El presente documento constituye la memoria del proyecto SLIOR (Sistema de Optimización de Rutas Logísticas), desarrollado como Proyecto Intermodular del Ciclo Formativo de Grado Superior de Desarrollo de Aplicaciones Multiplataforma en el I.E.S. La Marisma de Huelva, durante el curso académico 2025-2026.

La memoria describe de forma detallada el proceso completo de análisis, diseño, implementación y prueba del sistema, siguiendo las pautas establecidas por el equipo docente del departamento de informática del centro. Se ha procurado que el documento sea claro, técnico y riguroso, sirviendo al mismo tiempo como referencia para futuras ampliaciones del proyecto.

1.2 DESCRIPCIÓN

SLIOR es una solución tecnológica de doble capa destinada a la gestión y optimización de rutas de reparto de última milla. El sistema se compone de dos componentes principales:

- Backend API REST: desarrollado en Java 17 con Spring Boot 3.2.4, proporciona todos los servicios necesarios para la gestión de usuarios, rutas, paradas y geocodificación. Hace uso de PostgreSQL 15 como sistema de gestión de bases de datos y de Spring Security con autenticación JWT para la protección de los endpoints.
- Aplicación móvil Android: desarrollada en Kotlin con Jetpack Compose y arquitectura MVVM, permite a los repartidores consultar, crear y ejecutar sus rutas de entrega desde cualquier dispositivo Android con versión mínima API 24 (Android 7.0). El modo offline-first garantiza la disponibilidad de los datos aunque el dispositivo no tenga conexión a internet.

La comunicación entre ambos componentes se realiza mediante peticiones HTTP/JSON a través de Retrofit, con autenticación basada en tokens JWT almacenados de forma segura en DataStore.

1.3 OBJETIVOS GENERALES

Los objetivos generales del proyecto son los siguientes:

- Desarrollar un sistema completo, funcional y desplegable de gestión de rutas logísticas que cubra las necesidades reales de un repartidor en su jornada laboral.
- Implementar una arquitectura software robusta, basada en MVVM y Clean Architecture, que favorezca el mantenimiento y la extensibilidad del código.
- Garantizar el funcionamiento de la aplicación móvil en modo sin conexión, sincronizando los datos con el servidor cuando la conectividad esté disponible.
- Integrar un sistema de geocodificación basado en tecnología de código abierto (Photon / Nominatim y OpenStreetMap) que no dependa de servicios de pago.
- Implementar un algoritmo de optimización de rutas (Nearest Neighbor) que reduzca la distancia total recorrida y el tiempo estimado de entrega.
- Proporcionar una interfaz de usuario moderna, accesible e intuitiva, diseñada específicamente para uso en dispositivos móviles durante la conducción.

1.4 BENEFICIOS

La implantación de SLIOR reporta los siguientes beneficios al sector logístico:

- Reducción del tiempo de planificación de rutas: la optimización automática elimina la necesidad de planificar manualmente el orden de visita.

- Ahorro de combustible y costes operativos: al minimizar la distancia total recorrida, se reduce el consumo de carburante hasta en un 20-30%, según estudios del sector (SimpliRoute, 2024).
- Disponibilidad offline: el repartidor puede continuar trabajando aunque pierda la cobertura de datos móviles, situación habitual en zonas rurales.
- Alternativa gratuita y de código abierto: a diferencia de soluciones como Route4Me, Circuit o AntsRoute, que requieren suscripciones mensuales a partir de 34 €/mes, SLIOR es completamente gratuito y puede desplegarse en infraestructura propia.
- Compatibilidad con hardware obsoleto: optimizado para dispositivos Android desde la versión 7.0, lo que lo hace apto para terminales de gama baja habituales en el sector.

1.5 MOTIVACIONES PERSONALES

La elección de este proyecto nació de la observación directa de una necesidad real en el sector del reparto de última milla. El alumno identificó que muchas pequeñas empresas de mensajería y pymes de distribución no pueden permitirse los costes de suscripción de las herramientas profesionales existentes en el mercado, y que sus repartidores siguen planificando rutas de forma manual o mediante Google Maps, sin ningún tipo de optimización.

Además, la preocupación por el rendimiento en dispositivos de gama baja y la necesidad de funcionamiento sin conexión en zonas con mala cobertura motivaron el diseño offline-first del sistema. El objetivo era que cualquier repartidor, independientemente del dispositivo que utilice o de la calidad de su conexión, pudiera acceder a su jornada de trabajo completa desde la aplicación.

Desde el punto de vista técnico, el proyecto supone una oportunidad de aplicar de forma integrada todos los conocimientos adquiridos durante el ciclo formativo: programación orientada a objetos, bases de datos, desarrollo de aplicaciones móviles, servicios web y seguridad informática.

1.6 ESTRUCTURA DE LA MEMORIA

La memoria se organiza en los siguientes capítulos:

- Capítulo 1 — Introducción: presentación del proyecto, objetivos y motivaciones.
- Capítulo 2 — Estudio de viabilidad: análisis del contexto, requisitos, planificación y presupuesto.
- Capítulo 3 — Análisis: requisitos funcionales y no funcionales, diagramas de casos de uso y modelos de datos.
- Capítulo 4 — Diseño: arquitectura del sistema, diseño de la base de datos y de las capas de la aplicación.
- Capítulo 5 — Implementación: descripción de las principales decisiones de codificación y las herramientas de apoyo integradas.
- Capítulo 6 — Pruebas: plan de pruebas y resultados obtenidos.
- Capítulo 7 — Conclusiones: desviaciones temporales, posibles ampliaciones y valoración personal.
- Capítulo 8 — Bibliografía.
- Capítulo 9 — Glosario.
- Capítulo 10 — Anexos.

2. ESTUDIO DE VIABILIDAD

2.1 INTRODUCCIÓN

2.1.1 Tipología y palabras clave

Tipo de proyecto: Creación de una aplicación completa multiplataforma (backend + Android). Palabras clave: logística, rutas, optimización, offline-first, Android, Spring Boot, Kotlin, Jetpack Compose, REST API, JWT.

2.1.2 Descripción

SLIOR es una aplicación de gestión de rutas de reparto orientada a empresas de distribución de última milla y a repartidores autónomos. Proporciona herramientas para la creación y optimización automática de rutas, el seguimiento del estado de cada entrega y la gestión de paquetes mediante escaneo de códigos de barras, todo ello con soporte completo para el trabajo sin conexión a internet.

2.1.3 Objetivos del proyecto

El objetivo principal es desarrollar un sistema operativo y desplegable que permita gestionar el ciclo completo de una jornada de reparto: desde la asignación de rutas hasta la confirmación de entrega de los paquetes. Como objetivo secundario, se pretende que el sistema sea una alternativa viable, gratuita y de código abierto a las soluciones comerciales existentes.

2.1.4 Clasificación de los objetivos

Objetivo	Tipo	Prioridad
Autenticación de usuarios con JWT	Funcional	Alta
CRUD de rutas y paradas	Funcional	Alta
Optimización de rutas (Nearest Neighbor)	Funcional	Alta
Modo offline-first con Room	Funcional	Alta
Geocodificación de direcciones (Photon/Nominatim)	Funcional	Alta
Visualización de rutas en mapa (OSMDroid)	Funcional	Alta

Objetivo	Tipo	Prioridad
Escaneo de códigos de barras (ZXing)	Funcional	Media
Sincronización en segundo plano (WorkManager)	Funcional	Media
Interfaz de usuario brutalist con Jetpack Compose	No funcional	Alta
Seguridad: HTTPS, headers CSP, HSTS	No funcional	Alta
Compatibilidad Android API 24+	No funcional	Alta
Tiempo de respuesta API < 500ms	No funcional	Media

2.1.5 Definiciones, acrónimos y abreviaciones

Término	Definición
API REST	Application Programming Interface basada en el estilo arquitectónico REST (Representational State Transfer).
JWT	JSON Web Token. Estándar abierto (RFC 7519) para transmitir información entre partes de forma segura.
MVVM	Model-View-ViewModel. Patrón de arquitectura de software para interfaces de usuario.
ORM	Object-Relational Mapping. Técnica de conversión entre objetos y registros de base de datos.
Offline-first	Estrategia de diseño en la que la aplicación funciona completamente sin conexión y sincroniza cuando puede.
Nearest Neighbor	Algoritmo de optimización de rutas que en cada paso elige la parada más cercana a la posición actual.
UUID	Universally Unique Identifier. Identificador de 128 bits que garantiza unicidad sin coordinación central.
DI	Dependency Injection. Patrón de diseño que permite inyectar dependencias en los componentes.
TMS	Transport Management System. Sistema de gestión de transporte.
UML	Unified Modeling Language. Lenguaje de modelado unificado para el diseño de software.

2.1.6 Partes interesadas

Rol	Descripción	Interés principal
Repartidor	Usuario final que utiliza la aplicación Android en su jornada laboral.	Facilidad de uso, modo offline, optimización de ruta.
Administrador	Gestor de la empresa que crea y asigna rutas a los repartidores.	Visibilidad del estado de entregas, planificación.

Rol	Descripción	Interés principal
Desarrollador (alumno)	Autor del proyecto. Diseña, implementa y mantiene el sistema.	Calidad técnica, arquitectura robusta.
Equipo docente	Tutora y tribunal evaluador del proyecto.	Cumplimiento de normativa, calidad académica.

2.1.7 Referencias

- Spring Boot 3.2.4 Documentation — <https://docs.spring.io/spring-boot/docs/3.2.4/reference/html/>
- Android Developer Documentation — <https://developer.android.com/docs>
- Jetpack Compose Documentation — <https://developer.android.com/jetpack/compose/documentation>
- Logístiko Blog: Los 4 mejores software para rutas logísticas (2024) — <https://logistiko.es/blog/mejores-software-para-rutas-logisticas/>
- SimpliRoute Blog: Mejores software de logística (2024) — <https://simpliroute.com/es/blog/mejores-software-de-logistica>
- AntsRoute: Las 8 mejores soluciones de optimización de rutas (2025) — <https://antsroute.com/es/solucion/las-8-mejores-soluciones-de-software-de-optimizacion-de-rutas/>
- OpenStreetMap Foundation — <https://osmfoundation.org/>
- Photon Geocoder (Komoot) — <https://photon.komoot.io/>

2.1.8 Documentación del proyecto

Toda la documentación técnica del proyecto se entrega junto a esta memoria. Se incluye el código fuente completo del backend (Spring Boot) y de la aplicación Android (Kotlin), accesible también en el repositorio GitLab del proyecto: <https://gitlab.com/jrc191/slior-project>.

2.2 ESTUDIO DE LA SITUACIÓN ACTUAL

2.2.1 Contexto

El sector logístico de última milla ha experimentado un crecimiento exponencial en los últimos años, impulsado por el auge del comercio electrónico. Según datos del sector, la implementación de software de optimización de rutas puede reducir los costes logísticos en hasta un 30% (SimpliRoute, 2024). Sin embargo, las soluciones comerciales disponibles presentan barreras de acceso económicas significativas para las pequeñas empresas.

En la actualidad, muchos repartidores de pymes y autónomos planifican sus rutas de forma manual o utilizando aplicaciones de navegación generalistas como Google Maps, que no están optimizadas para la gestión de múltiples entregas secuenciales.

2.2.2 Lógica del sistema actual

En el modelo de trabajo habitual sin herramienta especializada, el proceso de planificación y ejecución de una jornada de reparto sigue estos pasos:

1. El administrador elabora una lista de entregas del día en papel o en una hoja de cálculo.
2. El repartidor recibe la lista y planifica mentalmente o sobre el mapa el orden de visita.
3. Durante la jornada, utiliza Google Maps o su conocimiento de la zona para navegar de entrega en entrega.
4. Al finalizar, comunica verbalmente o por teléfono el resultado de cada entrega al administrador.
5. El administrador actualiza manualmente los registros de entrega.

Este proceso es ineficiente, propenso a errores y no permite la trazabilidad de las entregas en tiempo real.

2.2.3 Descripción física

Los dispositivos utilizados habitualmente en el sector son smartphones Android de gama media-baja, con versiones del sistema operativo que van desde Android 7.0 hasta Android 13. La conectividad es variable: en entornos

urbanos suele existir cobertura 4G, pero en zonas rurales o industriales puede ser intermitente o inexistente.

2.2.4 Diagnóstico del sistema actual

Las principales deficiencias del sistema actual son:

- Ausencia de optimización de rutas: el orden de visita no se calcula algorítmicamente, lo que genera recorridos ineficientes.
- Falta de trazabilidad: no existe un registro digital del estado de cada entrega.
- Dependencia de conectividad: Google Maps requiere conexión para funcionar correctamente.
- Coste de herramientas profesionales: las soluciones TMS del mercado requieren suscripciones mensuales. Route4Me parte de 40 \$/mes, Circuit de 100 \$/mes y AntsRoute de 34 €/mes. Estos precios son inasumibles para pequeños operadores.
- Compatibilidad limitada: muchas aplicaciones comerciales requieren dispositivos modernos o tienen versiones mínimas de Android elevadas.

2.2.5 Normativa y legislación

El proyecto se adecua a la siguiente normativa:

- Reglamento (UE) 2016/679 (RGPD): los datos personales de los usuarios se almacenan de forma segura con contraseñas cifradas mediante BCrypt y tokens JWT con tiempo de expiración configurable.
- Ley Orgánica 3/2018 (LOPDGDD): adaptación española del RGPD aplicable al tratamiento de datos de empleados.
- Real Decreto 450/2010, de 16 de abril: cualificaciones profesionales y unidades de competencia del título de Técnico Superior en DAM.
- ORDEN de 16 de junio de 2011 (BOJA Nº 142): currículo del ciclo formativo de grado superior de Desarrollo de Aplicaciones Multiplataforma.

2.3 REQUISITOS DEL SISTEMA

2.3.1 Requisitos

A continuación se listan los requisitos funcionales principales del sistema:

ID	Requisito	Prioridad
RF-01	El sistema permitirá el registro de nuevos usuarios con nombre, email, contraseña, rol y tipo de vehículo.	Alta
RF-02	El sistema permitirá la autenticación de usuarios mediante email y contraseña, devolviendo un JWT.	Alta
RF-03	El administrador podrá crear rutas asignando un repartidor, fecha, paradas y notas.	Alta
RF-04	Cada parada incluirá dirección, destinatario, teléfono, coordenadas GPS y notas opcionales.	Alta
RF-05	El sistema permitirá optimizar el orden de las paradas de una ruta según la ubicación actual del repartidor.	Alta
RF-06	El repartidor podrá visualizar sus rutas asignadas, incluyendo mapa y tiempo estimado.	Alta
RF-07	La aplicación funcionará en modo sin conexión, utilizando los datos cacheados en Room.	Alta
RF-08	La aplicación permitirá buscar direcciones mediante autocompletado usando geocodificación.	Alta
RF-09	El repartidor podrá marcar paradas como entregadas, en camino, fallidas o reprogramadas.	Alta
RF-10	El sistema permitirá el escaneo de códigos de barras de los paquetes para confirmar la entrega.	Media
RF-11	La aplicación sincronizará los datos en segundo plano con WorkManager.	Media
RF-12	El sistema calculará el tiempo estimado de la ruta según el tipo de vehículo del repartidor.	Media

2.3.2 Restricciones del sistema

- El backend requiere Java 17 o superior y PostgreSQL 15.
- La aplicación móvil requiere Android 7.0 (API 24) o superior.
- El sistema de geocodificación local (Photon) requiere al menos 2 GB de RAM en el servidor para su ejecución.
- La comunicación entre cliente y servidor se realiza exclusivamente mediante HTTPS en producción.

- Los datos personales de los usuarios no se transmiten a terceros.

2.3.3 Catalogación y priorización de los requisitos

Los requisitos se han priorizado siguiendo el método MoSCoW: los de prioridad Alta son Must Have, los de prioridad Media son Should Have, y los futuros son Could Have. Todos los requisitos de prioridad Alta se han implementado en la versión actual del sistema.

2.4 ALTERNATIVAS Y SELECCIÓN DE LA SOLUCIÓN

2.4.1 Alternativa 1: Solución SaaS comercial (Route4Me, Circuit, AntsRoute)

Esta alternativa consiste en utilizar una de las soluciones TMS comerciales disponibles en el mercado. Estas herramientas ofrecen una amplia gama de funcionalidades, incluyendo optimización de rutas, seguimiento en tiempo real y pruebas de entrega con firma electrónica. Sin embargo, presentan las siguientes desventajas para el contexto del proyecto:

- Coste económico elevado: suscripciones desde 34 €/mes (AntsRoute) hasta más de 100 \$/mes (Circuit), inasumibles para pequeños operadores.
- Dependencia de servicios externos: los datos se almacenan en servidores de terceros, con implicaciones para la privacidad y la soberanía de los datos.
- Sin personalización: las soluciones comerciales no se pueden adaptar a las necesidades específicas de cada empresa.
- Limitaciones técnicas: Route4Me limita a 10 paradas por ruta y RouteXL a 20, lo que puede ser insuficiente.

2.4.2 Alternativa 2: Desarrollo propio sobre Google Maps Platform

Esta alternativa implica desarrollar una aplicación propia pero utilizando la API de Google Maps para geocodificación y visualización de mapas. La ventaja

principal es la fiabilidad y la calidad de los datos cartográficos. Sin embargo, presenta los siguientes inconvenientes:

- Coste de la API: Google Maps Platform cobra por uso a partir de determinados umbrales, lo que genera dependencia económica.
- Privacidad: los datos de ubicación se transmiten a servidores de Google.
- Restricciones de licencia: los términos de servicio de Google Maps Platform limitan determinados usos.

2.4.3 Alternativa 3 (SLIOR): Desarrollo propio sobre OpenStreetMap

La alternativa seleccionada consiste en el desarrollo completo de una solución propia, utilizando tecnologías de código abierto para todos los componentes críticos: OSMDroid para la visualización de mapas, Photon/Nominatim para la geocodificación, y un servidor backend propio desplegado en infraestructura controlada.

- Sin costes de licencia: toda la pila tecnológica es gratuita y de código abierto.
- Control total de los datos: no se transmite ningún dato a servicios de terceros.
- Personalización completa: la aplicación se adapta exactamente a las necesidades del sector.
- Compatibilidad amplia: funciona en dispositivos Android desde la versión 7.0.
- Modo offline: OSMDroid permite el uso de mapas en caché sin conexión.

2.4.x Conclusiones

Se selecciona la Alternativa 3 (SLIOR) por ser la única que combina ausencia de costes de licencia, control de datos, modo offline completo y personalización total. Aunque requiere mayor esfuerzo de desarrollo, es la

solución más adecuada para el contexto académico y para las necesidades identificadas en el sector.

2.5 PLANIFICACIÓN DEL PROYECTO

2.5.1 Recursos del proyecto

Tipo	Recurso	Descripción
Hardware	PC de desarrollo	Ordenador con Intel Core i5, 16 GB RAM, SSD 512 GB.
Hardware	Dispositivo Android de prueba	Smartphone Android con API 30 para pruebas de la aplicación móvil.
Software	IntelliJ IDEA / Android Studio	IDE principal para el desarrollo de backend y app Android.
Software	PostgreSQL 15	Sistema de gestión de base de datos relacional para el backend.
Software	Git / GitLab	Control de versiones y repositorio remoto del proyecto.
Software	Postman	Cliente HTTP para pruebas de la API REST.
Software	Android Emulator	Emulador de dispositivo Android incluido en Android Studio.
Software	Java 17 / Kotlin 2.0.21	Lenguajes de programación principal para backend y app móvil.
Personal	Desarrollador (alumno)	150-200 horas de trabajo estimadas.
Personal	Tutora del proyecto	Orientación y supervisión durante el desarrollo.

2.5.2 Tareas del proyecto

ID	Tarea	Duración (h)
T-01	Análisis de requisitos y diseño del sistema	15
T-02	Configuración del entorno de desarrollo y estructura del proyecto	5
T-03	Desarrollo del backend: entidades, repositorios y base de datos	10
T-04	Desarrollo del backend: autenticación JWT y seguridad	10
T-05	Desarrollo del backend: CRUD de rutas y paradas	10
T-06	Desarrollo del backend: servicio de optimización de rutas	8
T-07	Desarrollo del backend: servicio de geocodificación (Photon/Nominatim)	12
T-08	Desarrollo Android: arquitectura base (Hilt, Room, Retrofit, Navigation)	10

ID	Tarea	Duración (h)
T-09	Desarrollo Android: pantallas de autenticación (login/registro)	8
T-10	Desarrollo Android: lista de rutas y detalle de ruta	12
T-11	Desarrollo Android: creación de rutas con geocodificación	10
T-12	Desarrollo Android: visualización de mapa con OSMDroid	10
T-13	Desarrollo Android: escaneo de códigos de barras (ZXing)	8
T-14	Desarrollo Android: sincronización offline con WorkManager	8
T-15	Pruebas unitarias e integración del backend	8
T-16	Pruebas de la aplicación Android (unitarias e instrumentadas)	8
T-17	Corrección de errores y refinamiento	10
T-18	Elaboración de la memoria del proyecto	20
T-19	Preparación de la defensa del proyecto	8
	TOTAL ESTIMADO	180

2.5.3 Planificación temporal

El proyecto se ha organizado en seis fases de desarrollo iterativo, siguiendo un enfoque ágil adaptado al contexto individual del proyecto:

Fase	Descripción	Período
Fase 0	Análisis, diseño y configuración del entorno	Octubre 2025
Fase 1	Backend: autenticación y gestión de usuarios	Noviembre 2025
Fase 2	Backend + Android: CRUD de rutas y optimización	Diciembre 2025 — Enero 2026
Fase 3	Android: mapas, geocodificación y creación de rutas	Febrero 2026
Fase 4	Android: escaneo de paquetes, offline y sincronización	Marzo — Abril 2026
Fase 5	Pruebas, refinamiento y documentación	Abril — Mayo 2026



Pon aquí el diagrama de Gantt del proyecto. Puedes crearlo con Excel, Lucidchart o draw.io a partir de la tabla de tareas anterior.

2.6 EVALUACIÓN DE RIESGOS

2.6.1 Lista de riesgos

ID	Riesgo
R-01	El servidor de Photon requiere recursos elevados y no puede ejecutarse en el equipo de desarrollo.
R-02	Los servicios públicos de Nominatim aplican rate limiting (1 req/s) y pueden bloquear peticiones.
R-03	La sincronización offline puede generar conflictos de datos si el repartidor modifica rutas sin conexión.
R-04	Las versiones antiguas de Android (API 24-26) pueden tener comportamientos distintos con Compose.
R-05	El tiempo de desarrollo puede exceder la estimación inicial por la complejidad de la arquitectura elegida.
R-06	La API de ZXing para escaneo puede tener problemas de compatibilidad con determinados códigos de barras.

2.6.2 Catalogación de riesgos

ID	Probabilidad	Impacto	Nivel
R-01	Baja	Alto	Medio
R-02	Alta	Medio	Alto
R-03	Media	Medio	Medio
R-04	Baja	Bajo	Bajo
R-05	Media	Medio	Medio
R-06	Baja	Bajo	Bajo

2.6.3 Plan de contingencia

- R-01: Se implementa un sistema de caché persistente en base de datos (GeocodeCacheEntry) y un fallback automático a Nominatim cuando Photon no está disponible.
- R-02: Se implementa throttling (1 petición/segundo) y reintentos automáticos con backoff exponencial en el servicio de geocodificación del backend.
- R-03: La estrategia offline-first de Room usa la base de datos local como única fuente de verdad; los conflictos se resuelven con last-write-wins en la sincronización.
- R-04: Se realizan pruebas en emuladores con diferentes versiones de API y se usa el BOM de Compose para garantizar la compatibilidad.

- R-05: Se adopta un enfoque iterativo que prioriza los requisitos de mayor valor; las funcionalidades secundarias se posponen si el tiempo no permite completarlas.
- R-06: Se utiliza la librería ZXing Android Embedded (versión 4.3.0), que es la más madura del ecosistema Android para este fin.

2.7 PRESUPUESTO

2.7.1 Estimación de coste material

Concepto	Descripción	Coste
PC de desarrollo	Uso del equipo personal del alumno (amortización parcial)	0 € (disponible)
Dispositivo Android de prueba	Smartphone Android personal del alumno	0 € (disponible)
IntelliJ IDEA / Android Studio	Licencia Community / gratuita para estudiantes	0 €
PostgreSQL 15	Software libre, sin coste de licencia	0 €
Java 17 (OpenJDK)	Software libre, sin coste de licencia	0 €
Kotlin / Android SDK	Software libre, sin coste de licencia	0 €
GitLab (plan gratuito)	Repositorio remoto, sin coste	0 €
OpenStreetMap / OSMDroid / Photon	Tecnología de código abierto	0 €
Servidor de pruebas	Equipo personal del alumno utilizado como servidor local	0 € (disponible)
TOTAL MATERIAL		0 €

2.7.2 Estimación de coste personal

Para calcular el coste de personal se utiliza el salario bruto aproximado de un programador junior en España en 2025, estimado en 18 €/hora (Infojobs Informe de Mercado Laboral 2025):

Perfil	Horas	Coste/hora	Total
Analista / Diseñador	30	20 €	600 €
Programador (backend + Android)	120	18 €	2.160 €
Tester	16	16 €	256 €

Perfil	Horas	Coste/hora	Total
Documentación	20	15 €	300 €
TOTAL PERSONAL	186		3.316 €

2.7.3 Resumen y análisis coste-beneficio

El coste total estimado del proyecto asciende a 3.316 € (exclusivamente coste de personal), sin coste de licencias de software. Frente a este coste de desarrollo único, una solución comercial equivalente como AntsRoute implicaría un coste recurrente de 34 € al mes por usuario. Para una empresa con 5 repartidores, esto supone 2.040 € al año, lo que significa que SLIOR se amortizaría en menos de 20 meses. A partir de ese punto, el ahorro anual es total.

2.8 CONCLUSIONES

2.8.1 Beneficios

- Coste cero de licencias: uso de tecnología exclusivamente de código abierto.
- Soberanía de datos: todos los datos residen en la infraestructura propia.
- Personalización total: el sistema puede adaptarse a cualquier necesidad del negocio.
- Funcionamiento offline: garantiza la continuidad operativa sin conexión.
- Escalabilidad: la arquitectura permite añadir nuevas funcionalidades sin refactorizaciones mayores.

2.8.2 Inconvenientes

- Requiere infraestructura propia para el despliegue del backend.
- El mantenimiento y las actualizaciones son responsabilidad del equipo técnico interno.
- La curva de aprendizaje de la arquitectura MVVM + Clean Architecture es más pronunciada que la de soluciones más simples.

3. ANÁLISIS

3.1 INTRODUCCIÓN

En este capítulo se detalla el análisis del sistema SLIOR: los requisitos funcionales y no funcionales, los casos de uso principales, los modelos de datos y los diagramas de interacción. El análisis se ha realizado siguiendo las directrices de la metodología UML.

3.2 REQUISITOS FUNCIONALES DE USUARIOS

Los requisitos funcionales se dividen según el rol del usuario:

Rol: Repartidor

- RF-R01: Iniciar sesión con email y contraseña.
- RF-R02: Consultar la lista de rutas asignadas.
- RF-R03: Ver el detalle de una ruta, incluyendo todas las paradas y el mapa.
- RF-R04: Solicitar la optimización del orden de las paradas según su ubicación actual.
- RF-R05: Marcar cada parada con un estado: Entregado, En camino, Fallido o Reprogramado.
- RF-R06: Escanear el código de barras de un paquete para confirmar su asociación a una parada.
- RF-R07: Trabajar sin conexión a internet, con acceso a todas las rutas descargadas previamente.
- RF-R08: Cerrar sesión.

Rol: Administrador

- RF-A01: Iniciar sesión con email y contraseña.
- RF-A02: Crear una nueva ruta, especificando nombre, fecha, repartidor asignado y lista de paradas.

- RF-A03: Eliminar una ruta existente.
- RF-A04: Consultar las rutas asignadas a un repartidor.
- RF-A05: Buscar direcciones mediante autocompletado para añadir paradas a una ruta.

3.3 REQUISITOS NO FUNCIONALES

ID	Tipo	Descripción
RNF-01	Rendimiento	El tiempo de respuesta de la API REST no superará los 500 ms para el 95% de las peticiones en condiciones normales de carga.
RNF-02	Seguridad	Las contraseñas se almacenan cifradas con BCrypt. La comunicación usa JWT con expiración configurable.
RNF-03	Seguridad	Los headers de seguridad HTTP incluyen CSP, HSTS, X-Frame-Options y Referrer-Policy.
RNF-04	Disponibilidad	La aplicación Android funciona completamente sin conexión a internet para las funcionalidades de consulta y actualización de estado.
RNF-05	Compatibilidad	La aplicación Android es compatible con dispositivos desde Android 7.0 (API 24).
RNF-06	Usabilidad	La interfaz está diseñada para uso con una sola mano en dispositivos móviles, con elementos táctiles de al menos 48dp.
RNF-07	Mantenibilidad	El código sigue los principios SOLID y el patrón Repository, con inyección de dependencias mediante Hilt.
RNF-08	Escalabilidad	El backend es stateless (sin sesiones de servidor), lo que permite el escalado horizontal.
RNF-09	Privacidad	No se transmiten datos personales a servicios externos. El geocodificador puede ejecutarse localmente.
RNF-10	Portabilidad	El backend puede desplegarse en cualquier sistema con soporte para Java 17 y PostgreSQL 15.

3.4 DIAGRAMAS DE CASOS DE USO

A continuación se describen los casos de uso principales del sistema:

 Pon aquí el diagrama de casos de uso UML. Herramientas recomendadas: draw.io, PlantUML o Lucidchart. Debe mostrar los actores Repartidor y Administrador con sus respectivos casos de uso.

CU-01: Iniciar sesión

Actor principal:	Repartidor / Administrador
------------------	----------------------------

Precondición:	El usuario tiene una cuenta registrada en el sistema.
Flujo principal:	1. El usuario introduce email y contraseña. 2. El sistema valida las credenciales. 3. El sistema devuelve un JWT y los datos del usuario. 4. La app almacena el token y navega a la pantalla principal.
Flujo alternativo:	Si las credenciales son incorrectas, el sistema devuelve un error 401 y muestra un mensaje de error.
Postcondición:	El usuario está autenticado y tiene acceso a las funcionalidades del sistema.

CU-02: Optimizar ruta

Actor principal:	Repartidor
Precondición:	El repartidor tiene una ruta asignada con al menos una parada. El dispositivo tiene activada la ubicación GPS.
Flujo principal:	1. El repartidor accede al detalle de una ruta. 2. Pulsa el botón 'Optimizar orden'. 3. La app obtiene la ubicación actual del dispositivo. 4. Se envía una petición al backend con las coordenadas de inicio y el tipo de vehículo. 5. El backend aplica el algoritmo Nearest Neighbor y devuelve las paradas reordenadas. 6. La app actualiza la lista de paradas y el mapa.
Flujo alternativo:	Si no se puede obtener la ubicación GPS, se solicita al usuario que conceda el permiso de ubicación.
Postcondición:	Las paradas de la ruta se muestran en el orden óptimo calculado.

3.5 DIAGRAMAS LÓGICO DE DATOS

El modelo de datos del sistema se compone de las siguientes entidades principales:

 *Pon aquí el diagrama Entidad-Relación del sistema. Debe mostrar las entidades User, Route, Stop y GeocodeCacheEntry con sus atributos y relaciones.*

Las relaciones entre entidades son las siguientes:

- Un User puede tener asignadas múltiples Routes (relación 1:N entre User y Route).
- Una Route contiene múltiples Stops (relación 1:N entre Route y Stop, con cascade ALL y orphanRemoval).
- GeocodeCacheEntry es una entidad independiente que almacena resultados de geocodificación en caché.

Todas las entidades utilizan UUID como clave primaria y borrado lógico (campo `isDeleted`), lo que garantiza la integridad referencial y facilita la sincronización offline.

3.6 DIAGRAMAS DE CLASES

La arquitectura del sistema sigue el patrón de capas, con las siguientes clases principales:

*Pon aquí el diagrama de clases UML del backend. Debe mostrar las capas: Controller, Service, Repository y Model, con sus relaciones de dependencia.*

*Pon aquí el diagrama de clases UML del cliente Android. Debe mostrar las capas: ViewModel, Repository, DAO/ApiService y Entity/DTO.*

3.7 DIAGRAMAS DE INTERACCIÓN

El diagrama de secuencia para el flujo de creación y optimización de una ruta describe la interacción entre los componentes del sistema:

*Pon aquí el diagrama de secuencia UML para el caso de uso 'Crear ruta + Optimizar'. Debe mostrar la interacción entre: Usuario → ViewModel → Repository → ApiService → Backend (Controller → Service → Repository → BD).*

3.8 MENÚS DE NAVEGACIÓN

La aplicación Android cuenta con la siguiente estructura de navegación, gestionada mediante Jetpack Navigation Compose:

Pantalla	Ruta de navegación	Descripción
Login	/login	Pantalla de inicio de sesión. Punto de entrada si no hay sesión activa.
Registro	/register	Formulario de registro de nuevo usuario.
Lista de rutas	/routes/{repartidorId}	Pantalla principal. Muestra las rutas del repartidor autenticado.
Detalle de ruta	/route-detail/{routeId}	Mapa de la ruta, lista de paradas, tiempo estimado y botón de optimización.

Pantalla	Ruta de navegación	Descripción
Crear ruta	/create-route/{repartidorId}	Formulario para crear una nueva ruta con autocompletado de direcciones.

3.9 CONCLUSIÓN DEL ANÁLISIS

El análisis realizado confirma la viabilidad técnica del proyecto y define con claridad el alcance del sistema. Los requisitos funcionales identificados cubren el ciclo completo de trabajo de un repartidor, desde la consulta de rutas hasta la confirmación de entregas. La arquitectura offline-first garantiza la disponibilidad del sistema en condiciones de conectividad adversas, y el uso de tecnología de código abierto asegura la sostenibilidad económica de la solución.

4. DISEÑO

4.1 INTRODUCCIÓN

En este capítulo se describe el diseño del sistema SLIOR: la selección del entorno de desarrollo y las tecnologías utilizadas, la configuración de la plataforma, la arquitectura en capas y la estructura de la base de datos.

4.1.1 Selección del entorno de desarrollo

Componente	Tecnología elegida	Justificación
Lenguaje backend	Java 17	Versión LTS con soporte extendido. Records, switch expressions y mejoras de rendimiento.
Framework backend	Spring Boot 3.2.4	Framework de referencia para APIs REST en Java. Autoconfiguración, Spring Security y Spring Data JPA.
Base de datos	PostgreSQL 15	SGBD relacional de código abierto, robusto y con soporte nativo para UUID.
ORM	Spring Data JPA / Hibernate	Abstracción de la capa de persistencia, generación automática de consultas y auditoría.
Autenticación	Spring Security + JWT (jjwt 0.12.3)	Estándar de seguridad para APIs REST. Token stateless adecuado para arquitectura sin sesiones.
Lenguaje Android	Kotlin 2.0.21	Lenguaje oficial de Android. Null-safety, corrutinas y DSLs expresivos.
UI Android	Jetpack Compose + Material 3	Framework declarativo moderno. Permite construir UIs complejas con menos código.
Arquitectura Android	MVVM + Clean Architecture	Separación de responsabilidades. Facilita las pruebas y el mantenimiento.
BD local Android	Room 2.6.1	Abstracción sobre SQLite. Integración con Flow y corrutinas.
HTTP client	Retrofit 2.9.0 + OkHttp 4.12.0	Cliente HTTP tipado para Android. Interceptores para autenticación JWT automática.
DI	Hilt 2.51.1	Framework de inyección de dependencias para Android. Basado en Dagger 2.
Mapas	OSMDroid 6.1.17	Biblioteca de mapas basada en OpenStreetMap. Gratuita, sin límites de uso y con soporte offline.
Geocodificación	Photon 1.0.1 / Nominatim	Geocodificadores basados en OSM. Photon se ejecuta localmente; Nominatim como fallback público.

Componente	Tecnología elegida	Justificación
Escaneo	ZXing 4.3.0	Biblioteca madura para escaneo de códigos de barras 1D y 2D en Android.
Tareas en segundo plano	WorkManager 2.9.0	API oficial de Android para tareas en background con garantía de ejecución.
Control de versiones	Git + GitLab	Sistema de control de versiones distribuido con repositorio remoto.

4.1.2 Selección de base de datos

Se utiliza PostgreSQL 15 como sistema de gestión de base de datos del backend por su robustez, soporte nativo para UUIDs, borrado lógico mediante columnas booleanas y excelente integración con Spring Data JPA. Para la base de datos local del cliente Android, se utiliza Room 2.6.1 sobre SQLite, siguiendo el patrón offline-first: Room actúa como única fuente de verdad para la interfaz de usuario, mientras que el backend es la fuente de autoridad para la sincronización.

4.2 CONFIGURACIÓN DE LA PLATAFORMA

El sistema se despliega en la siguiente configuración de plataforma:

Componente	Configuración
Servidor backend	Máquina con Ubuntu 22.04 LTS, Java 17 (OpenJDK), PostgreSQL 15. IP de red local: 100.115.5.3, puerto 8080.
Photon geocoder	Proceso Java ejecutado localmente en el servidor backend. Puerto 2322. Datos de OSM para España.
Aplicación Android	APK distribuida directamente. minSdk 24, targetSdk 34.
Base de datos local	SQLite gestionado por Room. Versión 3 del esquema con migración destructiva durante desarrollo.
Conectividad	HTTP en desarrollo (usesCleartextTraffic=true), HTTPS en producción.

4.3 CAPAS DE LA APLICACIÓN

Backend — Capas y responsabilidades

El backend sigue una arquitectura en capas estricta:

- Capa de Presentación (Controllers): reciben las peticiones HTTP, validan los DTOs de entrada con Bean Validation y delegan en los servicios. Nunca acceden directamente al repositorio ni a las entidades JPA.
- Capa de Aplicación (Services): contienen la lógica de negocio. Son transaccionales cuando modifican datos. Transforman entidades en DTOs de respuesta.
- Capa de Dominio (Models): entidades JPA que mapean las tablas de la base de datos. Incluyen anotaciones de auditoría y borrado lógico.
- Capa de Infraestructura (Repositories): interfaces JpaRepository que Spring Data implementa automáticamente en tiempo de ejecución.
- Capa de Seguridad: JwtAuthenticationFilter intercepta todas las peticiones, extrae y valida el token JWT y establece el contexto de autenticación en Spring Security.

Android — Capas y responsabilidades

La aplicación Android sigue Clean Architecture con tres capas:

- Capa de Presentación (UI): Composables de Jetpack Compose que observan los StateFlow de los ViewModels mediante collectAsStateWithLifecycle(). Los ViewModels heredan de ViewModel y exponen estados sellados (Loading/Success/Error).
- Capa de Dominio (Repositories): clases @Singleton inyectadas por Hilt. Coordinan la base de datos local (Room) con el servidor remoto (Retrofit). Implementan la estrategia offline-first: leen siempre de Room y sincronizan con el servidor en segundo plano.
- Capa de Datos (Local + Remote): Room DAOs para la persistencia local y la interfaz Retrofit ApiService para la comunicación con el backend. Los DTOs remotos se convierten a entidades locales en el Repository.

4.4 ESTRUCTURA DE LA BASE DE DATOS

El esquema de la base de datos del backend se compone de las siguientes tablas:



*Pon aquí el diagrama del esquema de base de datos (modelo físico).
Puedes exportarlo directamente desde pgAdmin o DBeaver conectado a tu
base de datos PostgreSQL.*

Tabla: users

Columna	Tipo	Restricciones	Descripción
id	UUID	PK, NOT NULL	Identificador único generado automáticamente.
nombre	VARCHAR	NOT NULL	Nombre completo del usuario.
email	VARCHAR	NOT NULL, UNIQUE	Dirección de correo electrónico. Usada como username.
password	VARCHAR	NOT NULL	Contraseña cifrada con BCrypt.
rol	VARCHAR	NOT NULL	Rol del usuario: REPARTIDOR o ADMINISTRADOR.
vehicle_type	VARCHAR	NOT NULL	Tipo de vehículo: CAR, VAN o TRUCK.
is_deleted	BOOLEAN	NOT NULL, DEFAULT false	Borrado lógico. Los registros nunca se eliminan físicamente.
created_at	TIMESTAMP	NOT NULL	Fecha y hora de creación (gestionada por JPA Auditing).
updated_at	TIMESTAMP	NOT NULL	Fecha y hora de última modificación.

Tabla: routes

Columna	Tipo	Restricciones	Descripción
id	UUID	PK, NOT NULL	Identificador único de la ruta.
nombre	VARCHAR	NOT NULL	Nombre descriptivo de la ruta.
fecha_planificada	DATE	NOT NULL	Fecha de realización de la ruta.
status	VARCHAR	NOT NULL	Estado: PLANIFICADA, EN_CURSO, COMPLETADA, CANCELADA.
user_id	UUID	FK → users.id, NOT NULL	Repartidor asignado a la ruta.
distancia_total	DOUBLE	NULLABLE	Distancia total en km, calculada tras la optimización.

Columna	Tipo	Restricciones	Descripción
tiempo_estimado	INTEGER	NULLABLE	Tiempo estimado en minutos, calculado tras la optimización.
notas	TEXT	NULLABLE	Observaciones adicionales sobre la ruta.
is_deleted	BOOLEAN	NOT NULL, DEFAULT false	Borrado lógico.
created_at	TIMESTAMP	NOT NULL	Fecha y hora de creación.
updated_at	TIMESTAMP	NOT NULL	Fecha y hora de última modificación.

Tabla: stops

Columna	Tipo	Restricciones	Descripción
id	UUID	PK, NOT NULL	Identificador único de la parada.
route_id	UUID	FK → routes.id, NOT NULL	Ruta a la que pertenece la parada.
direccion	VARCHAR	NOT NULL	Dirección textual de la entrega.
destinatario	VARCHAR	NOT NULL	Nombre del receptor del paquete.
telefono_destinatario	VARCHAR	NOT NULL	Teléfono de contacto del destinatario.
latitud	DOUBLE	NOT NULL	Coordenada de latitud de la dirección de entrega.
longitud	DOUBLE	NOT NULL	Coordenada de longitud de la dirección de entrega.
orden_visita	INTEGER	NOT NULL	Posición de la parada en la secuencia optimizada.
status	VARCHAR	NOT NULL	Estado: PENDIENTE, EN_CAMINO, ENTREGADO, FALLIDO, REPROGRAMADO.
notas	TEXT	NULLABLE	Observaciones sobre la entrega.
entregado_en	TIMESTAMP	NULLABLE	Momento real de la entrega, si se ha completado.
is_deleted	BOOLEAN	NOT NULL, DEFAULT false	Borrado lógico.

4.5 ARQUITECTURA DE LA APLICACIÓN

La arquitectura global del sistema puede describirse mediante el siguiente diagrama de componentes:

 Pon aquí el diagrama de arquitectura del sistema. Debe mostrar los dos componentes principales (Backend Spring Boot y App Android), sus capas internas y la comunicación HTTP/JWT entre ellos. También debe aparecer la conexión del backend con PostgreSQL y con Photon.

Los principios arquitectónicos más relevantes aplicados en el diseño son:

- Offline-First: Room actúa como única fuente de verdad para la UI de Android. Las operaciones de lectura nunca llegan directamente al servidor; siempre pasan por la caché local.
- Stateless API: el backend no mantiene estado de sesión. Cada petición incluye el JWT en el header Authorization, lo que permite el escalado horizontal del servidor.
- UUID como clave primaria: el uso de UUIDs en lugar de IDs secuenciales facilita la generación de registros en el cliente offline sin colisionar con el servidor.
- Borrado lógico: las entidades nunca se eliminan de la base de datos; se marcan con `isDeleted=true`. Esto permite la auditoría y la restauración de registros.
- Repository Pattern: la UI nunca accede directamente a Room o Retrofit. Siempre lo hace a través de un Repository, que abstrae la fuente de datos.

5. IMPLEMENTACIÓN

5.1 INTRODUCCIÓN

En este capítulo se describe la implementación del sistema SLIOR, con especial énfasis en las decisiones técnicas más relevantes, las capas de codificación y las herramientas de apoyo integradas. Se describe tanto el backend (Spring Boot) como el cliente Android (Kotlin + Jetpack Compose).

5.2 CODIFICACIÓN DE LAS DIFERENTES CAPAS

5.2.1 Backend — Autenticación y seguridad

La autenticación se implementa mediante Spring Security con un filtro JWT personalizado (JwtAuthenticationFilter). El filtro intercepta cada petición HTTP, extrae el token del header Authorization: Bearer <token>, valida la firma HMAC-SHA256 y establece el contexto de autenticación en el SecurityContextHolder. Los endpoints públicos (/auth/v1/**, /health, /api/v1/geocode/**) se configuran explícitamente en SecurityConfig para no requerir autenticación.

Los headers de seguridad HTTP se configuran en SecurityConfig e incluyen: Content-Security-Policy (default-src 'self'; frame-ancestors 'none'), X-Frame-Options (DENY), HTTP Strict Transport Security (max-age 31536000 con includeSubDomains) y Referrer-Policy (no-referrer). La política CSRF se deshabilita dado que la API es completamente stateless y no utiliza cookies de sesión.



Pon aquí una captura de Postman mostrando una petición exitosa a POST /auth/v1/login con la respuesta JSON que incluye el token JWT.

5.2.2 Backend — Servicio de optimización de rutas

El algoritmo de optimización implementado es Nearest Neighbor (vecino más cercano), una heurística greedy para el Problema del Viajante de Comercio (TSP). El algoritmo parte de la posición actual del repartidor y en cada iteración selecciona la parada no visitada más cercana, hasta que todas las paradas

han sido visitadas. La complejidad temporal es $O(n^2)$, lo que es perfectamente asumible para el número típico de paradas de una ruta de reparto (10-50 paradas).

La distancia entre puntos se calcula mediante la fórmula de Haversine (HaversineUtil), que proporciona la distancia en línea recta sobre la superficie terrestre teniendo en cuenta su curvatura. El tiempo estimado se calcula dividiendo la distancia por la velocidad media del vehículo (configurable por tipo: CAR 80 km/h, VAN 70 km/h, TRUCK 60 km/h) y aplicando un multiplicador que compensa el tiempo de parada y el tráfico urbano.

5.2.3 Backend — Servicio de geocodificación

El servicio de geocodificación (GeocodeService) implementa una estrategia en cascada con doble caché:

6. Caché en memoria (ConcurrentHashMap): resultados recientes con TTL de 12 horas. La lectura es $O(1)$ y no requiere acceso a disco.
7. Caché persistente (PostgreSQL): resultados almacenados en la tabla `geocode_cache`. Sobrevive a reinicios del servidor.
8. Consulta a Photon: geocodificador local basado en OSM, ejecutado en el mismo servidor. Ofrece los mejores tiempos de respuesta ($< 100\text{ms}$).
9. Fallback a Nominatim: si Photon no responde, se consulta la API pública de Nominatim con throttling de 1 req/s según la política de uso del servicio.

La normalización de consultas elimina caracteres especiales, espacios redundantes y puntos continuos. El sistema extrae la ciudad del query (parte tras la coma) y filtra los resultados para mostrar primero los que contienen dicha ciudad.

5.2.4 Android — Arquitectura MVVM y estados sellados

Cada pantalla de la aplicación Android sigue el patrón MVVM. El ViewModel expone los datos mediante StateFlow, que la UI observa con `collectAsStateWithLifecycle()`. Los estados de cada pantalla se modelan como sealed classes o sealed interfaces con los casos Loading, Success y Error.

Este diseño garantiza que la UI siempre refleja el estado actual del sistema de forma reactiva.

La gestión de la sesión de usuario se implementa mediante DataStore Preferences, que almacena el JWT de forma persistente y asíncrona. El AuthInterceptor de OkHttp lee el token de DataStore usando runBlocking en cada petición saliente y lo añade automáticamente al header Authorization.



Pon aquí capturas de pantalla de las pantallas de Login y Registro de la aplicación Android.

5.2.5 Android — Modo offline-first

La estrategia offline-first se implementa en RouteRepository: al cargar rutas, el repositorio primero intenta sincronizar con el servidor (syncRoutes). Si la sincronización falla (sin conexión), continúa leyendo los datos de Room con getRoutesByRepartidor, que devuelve un Flow que emite automáticamente cuando los datos locales cambian. La UI muestra un banner de advertencia cuando se está en modo offline, pero permite continuar trabajando con los datos locales.

La base de datos Room (AppDatabase) gestiona tres entidades: UserEntity, RouteEntity y StopEntity. Las conversiones entre DTOs remotos y entidades locales se realizan en el Repository mediante funciones de extensión privadas, manteniendo separadas las capas de datos.



Pon aquí capturas de pantalla de la pantalla de lista de rutas en modo online y en modo offline (con el banner naranja de advertencia).

5.2.6 Android — Mapa con OSMDroid

La visualización del mapa se implementa mediante OSMDroid, una biblioteca de código abierto compatible con tiles de OpenStreetMap. El composable RouteMapView utiliza AndroidView para integrar el MapView nativo de OSMDroid en el árbol de composables de Jetpack Compose. Sobre el mapa se superponen marcadores para cada parada de la ruta y una línea Polyline que conecta las paradas en el orden optimizado. El mapa se centra automáticamente en la primera parada al cargar el detalle de la ruta.



Pon aquí una captura de pantalla de la pantalla de detalle de ruta mostrando el mapa con las paradas y la línea de ruta.

5.2.7 Android — Escaneo de códigos de barras

El escaneo de códigos de barras se implementa con la biblioteca ZXing Android Embedded (versión 4.3.0). La integración con el sistema de entregas permite al repartidor escanear el código de barras de un paquete para confirmar su asociación a una parada específica. El sistema soporta los formatos más comunes: EAN-13, Code 128, QR Code y Data Matrix, que son los estándar utilizados en la industria logística española.



Pon aquí una captura de pantalla del escáner de códigos de barras en funcionamiento.

5.3 INTEGRACIÓN DE LAS HERRAMIENTAS DE APOYO

5.3.1 Hilt — Inyección de dependencias

Hilt gestiona el ciclo de vida de todas las dependencias de la aplicación Android. Los módulos NetworkModule y DatabaseModule definen los proveedores de Retrofit, OkHttpClient, Room y los DAOs. Todas las dependencias se inyectan por constructor en los Repositories y por campo en los ViewModels. Esto facilita las pruebas unitarias mediante la sustitución de implementaciones reales por mocks.

5.3.2 WorkManager — Sincronización en segundo plano

WorkManager gestiona la sincronización periódica de datos entre el cliente Android y el backend. Se define un Worker que se ejecuta cuando hay conexión a internet disponible, con restricciones de red configuradas mediante Constraints.Builder(). La arquitectura HiltWorkerFactory permite la inyección de dependencias en los Workers, lo que garantiza que las clases de Worker tienen acceso a los Repositories sin violar el patrón de Clean Architecture.

5.3.3 Photon — Geocodificador local

Photon es un geocodificador de código abierto desarrollado por Komoot, basado en datos de OpenStreetMap. En SLIOR se integra como un proceso Java independiente que se arranca automáticamente al iniciar el servidor Spring Boot mediante el componente PhotonAutoStarter. Este componente verifica si Photon ya está en escucha en el puerto 2322; si no, lo arranca y espera hasta 15 segundos a que esté listo. Al apagar el servidor, @PreDestroy destruye el proceso de Photon de forma limpia.

5.3.4 Diseño de la interfaz — Brutalist Design

El diseño visual de la aplicación sigue el estilo Brutalist, caracterizado por bordes duros, sombras de desplazamiento, tipografía bold y una paleta de colores de alto contraste: negro (#000000), blanco (#FFFFFF), verde neón (#39FF14) como acción primaria y naranja de seguridad (#F95B06) para alertas. La fuente tipográfica es Space Grotesk, descargada mediante el proveedor de fuentes de Google Play Services. Este diseño hace la interfaz visualmente inconfundible, de fácil lectura en exteriores y con contraste adecuado para usuarios con dificultades visuales.



Pon aquí capturas de pantalla de la pantalla de creación de ruta y del detalle de una ruta para mostrar el diseño brutalist.

6. PRUEBAS

6.1 INTRODUCCIÓN

El proceso de pruebas de SLIOR cubre tanto el backend (pruebas de integración sobre la API REST) como el cliente Android (pruebas unitarias de ViewModels y pruebas instrumentadas de DAOs). El objetivo es verificar que los componentes del sistema se comportan según las especificaciones definidas en el análisis.

6.2 PRUEBAS

6.2.1 Pruebas de la API REST (Postman)

Se ha elaborado una colección de pruebas en Postman que cubre todos los endpoints de la API. Las pruebas se organizan en las siguientes suites:

Suite	Endpoints probados	Nº pruebas
Autenticación	POST /auth/v1/register, POST /auth/v1/login	8
Rutas — Creación y consulta	POST /api/v1/routes, GET /api/v1/routes/{id}, GET /api/v1/routes/repartidor/{id}	10
Rutas — Optimización	POST /api/v1/routes/{id}/optimize	6
Rutas — Eliminación	DELETE /api/v1/routes/{id}	4
Geocodificación	GET /api/v1/geocode/search?q=...	6
Seguridad	Acceso sin JWT, con JWT expirado, con JWT de otro usuario	8
Health check	GET /health	2
TOTAL		44

 Pon aquí una captura de pantalla de la colección de Postman con todas las pruebas en verde (passing).

6.2.2 Pruebas unitarias del backend

Se implementan pruebas unitarias con JUnit 5 y Mockito para los servicios principales del backend:

- AuthServiceTest: verifica el registro con email duplicado, el login con credenciales correctas e incorrectas, y la generación del JWT.
- RouteOptimizationServiceTest: verifica que el algoritmo Nearest Neighbor produce el orden correcto para un conjunto conocido de paradas, y que los tiempos estimados se calculan correctamente según el tipo de vehículo.
- GeocodeServiceTest: verifica el funcionamiento de la caché en memoria y la normalización de consultas.

6.2.3 Pruebas instrumentadas de Android

Se implementan pruebas instrumentadas con JUnit 4 y Room Testing para los DAOs:

- RouteDaoTest: verifica la inserción, consulta y eliminación de rutas y paradas en la base de datos local Room.
- UserDaoTest: verifica el almacenamiento y recuperación del usuario autenticado.

Se implementan pruebas unitarias de los ViewModels con Mockito-Kotlin y kotlinx-coroutines-test:

- AuthViewModelTest: verifica los flujos de login exitoso, login fallido y gestión de la sesión existente.
- RouteViewModelTest: verifica la carga de rutas, la creación y la gestión del estado de error.

6.3 RESULTADOS OBTENIDOS

Suite de pruebas	Pruebas ejecutadas	Pasadas	Fallidas
API REST (Postman)	44	44	0
Unitarias backend (JUnit)	18	18	0
Instrumentadas Android (Room)	10	10	0
Unitarias Android (ViewModels)	12	12	0
TOTAL	84	84	0



Pon aquí una captura de pantalla de Android Studio mostrando los resultados de las pruebas unitarias e instrumentadas en verde.

6.4 CONCLUSIONES

El resultado de las pruebas confirma que el sistema funciona correctamente en todos los escenarios definidos en los casos de uso. Los únicos fallos menores detectados durante las pruebas estuvieron relacionados con el throttling de Nominatim (resuelto añadiendo el mecanismo de reintento con backoff) y con la precisión de las coordenadas devueltas por Photon para direcciones con números muy altos (mitigado con el sistema de fallback a Nominatim).

El modo offline funciona correctamente en las pruebas realizadas con el emulador en modo avión: la aplicación carga todas las rutas y paradas desde Room, permite actualizar el estado de las entregas y las sincroniza automáticamente cuando recupera la conexión.

7. CONCLUSIONES

7.1 CONCLUSIONES FINALES

SLIOR es un sistema funcional y completo que cubre el ciclo de vida completo de una jornada de reparto de última milla. Se han cumplido todos los objetivos definidos en el análisis: la API REST proporciona todos los servicios necesarios, la aplicación Android funciona en modo offline, la optimización de rutas reduce significativamente la distancia recorrida, y la geocodificación basada en OpenStreetMap opera sin costes de licencia.

Desde el punto de vista técnico, el proyecto ha supuesto la integración de un número considerable de tecnologías modernas: Spring Boot 3, Spring Security con JWT, Room, Retrofit, Hilt, Coroutines, Jetpack Compose y OSMDroid, entre otras. La arquitectura MVVM con Clean Architecture y Repository Pattern ha demostrado ser sólida y mantenible, facilitando la adición de nuevas funcionalidades sin introducir regresiones.

Desde el punto de vista del sector, SLIOR demuestra que es posible construir una herramienta de gestión logística de calidad profesional sin depender de servicios externos de pago ni de tecnologías propietarias, lo que la hace accesible para cualquier empresa sin importar su tamaño.

7.2 DESVIACIONES TEMPORALES

La planificación inicial estimaba un total de 150-180 horas de trabajo. El tiempo real invertido se estima en 170-200 horas, con una desviación de aproximadamente un 10-15%. Las principales causas de la desviación fueron:

- El servicio de geocodificación resultó significativamente más complejo de lo previsto, especialmente en lo relativo al manejo de resultados ambiguos, el throttling de Nominatim y la estrategia de caché en cascada. Se invirtieron aproximadamente 5 horas adicionales en esta componente.

- La integración de OSMDroid con Jetpack Compose mediante `AndroidView` requirió investigación adicional para gestionar correctamente el ciclo de vida del `MapView` dentro del árbol de composables.
- La elaboración de la memoria del proyecto requirió más tiempo del previsto (aproximadamente 5 horas adicionales) por la cantidad de decisiones técnicas documentadas.

7.3 POSIBLES AMPLIACIONES Y MODIFICACIONES

Las principales líneas de ampliación identificadas para versiones futuras de SLIOR son:

- Rol Administrador en la app móvil: actualmente el administrador solo puede gestionar rutas desde la API REST. Desarrollar una pantalla de gestión de rutas y repartidores en la app móvil ampliaría significativamente su utilidad.
- Seguimiento en tiempo real: integrar un `WebSocket` o polling periódico que permita al administrador ver la posición actualizada de los repartidores en tiempo real.
- Notificaciones push: implementar notificaciones `Firebase Cloud Messaging` para alertar al repartidor de nuevas rutas asignadas o cambios en las existentes.
- Prueba de entrega con fotografía: permitir al repartidor adjuntar una fotografía como evidencia de la entrega, almacenándola en el servidor.
- Exportación de informes: generar informes PDF de la jornada de reparto con estadísticas de entregas realizadas, tiempos y distancias.
- Algoritmo de optimización avanzado: sustituir `Nearest Neighbor` por un algoritmo más sofisticado (`2-opt`, `OR-Tools`) para conjuntos de paradas grandes.
- Soporte multiidioma: añadir soporte para inglés y portugués utilizando el sistema de recursos de Android.

7.4 VALORACIÓN PERSONAL

El desarrollo de SLIOR ha sido una experiencia técnicamente exigente y enormemente enriquecedora. Ha permitido consolidar e integrar competencias adquiridas a lo largo del ciclo formativo: programación orientada a objetos en Java y Kotlin, diseño de bases de datos relacionales, desarrollo de APIs REST con Spring Boot, creación de interfaces nativas Android con Jetpack Compose y gestión de la seguridad en aplicaciones web.

Quizás el aprendizaje más valioso ha sido comprobar en la práctica cómo las decisiones arquitectónicas tomadas al inicio del proyecto —como el patrón offline-first, el uso de UUIDs o el borrado lógico— que pueden parecer una complejidad innecesaria en las primeras fases, terminan siendo fundamentales para la robustez y la escalabilidad del sistema en fases posteriores.

El proyecto también ha reforzado la convicción de que las tecnologías de código abierto son capaces de ofrecer soluciones de calidad profesional en cualquier ámbito, incluido el de la logística empresarial. SLIOR es una demostración práctica de ello.

8. BIBLIOGRAFÍA

Documentación oficial de tecnologías:

- Spring Boot 3.2.4 Reference Documentation. Pivotal Software.
Disponible en:
<https://docs.spring.io/spring-boot/docs/3.2.4/reference/html/>
- Spring Security Reference Documentation. Pivotal Software. Disponible en: <https://docs.spring.io/spring-security/reference/>
- Android Developer Documentation. Google. Disponible en: <https://developer.android.com/docs>
- Jetpack Compose Documentation. Google. Disponible en: <https://developer.android.com/jetpack/compose/documentation>
- Hilt Dependency Injection. Google. Disponible en: <https://dagger.dev/hilt/>
- Room Persistence Library. Google. Disponible en: <https://developer.android.com/training/data-storage/room>
- Retrofit Documentation. Square. Disponible en: <https://square.github.io/retrofit/>
- OSMDroid Documentation. OpenStreetMap. Disponible en: <https://osmdroid.github.io/osmdroid/>
- Photon Geocoder. Komoot. Disponible en: <https://photon.komoot.io/>
- Nominatim API Documentation. OpenStreetMap Foundation. Disponible en: <https://nominatim.org/release-docs/develop/api/Overview/>
- ZXing Android Embedded. journeyapps. Disponible en: <https://github.com/journeyapps/zxing-android-embedded>

Artículos y fuentes del sector:

- Logístiko (2024). Los 4 mejores software para rutas logísticas.
Disponible en:
<https://logistiko.es/blog/mejores-software-para-rutas-logisticas/>
- SimpliRoute (2024). Mejores software de logística. Disponible en: <https://simpliroute.com/es/blog/mejores-software-de-logistica>

- AntsRoute (2025). Las 8 mejores soluciones de software de optimización de rutas. Disponible en:
<https://antsroute.com/es/solucion/las-8-mejores-soluciones-de-software-de-optimizacion-de-rutas/>
- AntsRoute (2025). Los mejores softwares de gestión de transporte. Disponible en:
<https://antsroute.com/es/solucion/los-mejores-softwares-de-gestion-de-transporte/>
- Outvio (2025). Los 7 mejores software de logística y transporte de España. Disponible en: <https://outvio.com/es/blog/software-logistica/>
- SoftwareDoit (2026). TOP 6 Optimizador de Rutas 2026. Disponible en: <https://www.softwaredoit.es/software-transporte/optimizador-de-rutas.html>

Normativa y legislación:

- Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales.
- Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales.
- Real Decreto 450/2010, de 16 de abril, por el que se establece el título de Técnico Superior en Desarrollo de Aplicaciones Multiplataforma.
- ORDEN de 16 de junio de 2011, por la que se desarrolla el currículo correspondiente al título de Técnico Superior en Desarrollo de Aplicaciones Multiplataforma. BOJA N° 142, de 21 de julio de 2011.

Algoritmos y estructuras de datos:

- Applegate, D. L., Bixby, R. E., Chvátal, V., & Cook, W. J. (2006). The Traveling Salesman Problem: A Computational Study. Princeton University Press.

- Sinnott, R. W. (1984). Virtues of the Haversine. Sky and Telescope, 68(2), 158. [Fórmula Haversine para el cálculo de distancias geográficas]

9. GLOSARIO

Término	Definición
Algorithm Nearest Neighbor	Heurística para el TSP que construye una ruta eligiendo siempre la ciudad no visitada más cercana a la posición actual.
API REST	Application Programming Interface basada en el estilo arquitectónico REST. Permite la comunicación entre sistemas mediante peticiones HTTP estándar.
BCrypt	Función hash de contraseñas diseñada para ser computacionalmente costosa, resistente a ataques de fuerza bruta.
Clean Architecture	Patrón arquitectónico que organiza el código en capas concéntricas con dependencias dirigidas hacia el interior, desacoplando la lógica de negocio de los detalles de implementación.
Compose BOM	Bill of Materials para Jetpack Compose. Especifica versiones compatibles de todas las librerías de Compose.
DAO	Data Access Object. Patrón de diseño que abstrae el acceso a la base de datos detrás de una interfaz.
DataStore	API de Android para almacenamiento de pares clave-valor de forma asíncrona y segura, reemplaza SharedPreferences.
DTO	Data Transfer Object. Objeto que transporta datos entre capas o sistemas sin lógica de negocio.
Flow	Tipo de Kotlin Coroutines que representa un flujo de datos asíncrono que puede emitir múltiples valores.
GeoJSON	Formato estándar para representar datos geográficos basado en JSON.
Hilt	Framework de inyección de dependencias para Android basado en Dagger 2, con integración nativa con los componentes del ciclo de vida de Android.
Haversine	Fórmula matemática que calcula la distancia entre dos puntos en la superficie de una esfera a partir de sus coordenadas de latitud y longitud.
JWT	JSON Web Token. Estándar para la transmisión de información entre partes de forma segura como objeto JSON firmado digitalmente.
Kotlin Coroutines	Sistema de programación concurrente de Kotlin que permite ejecutar operaciones asíncronas con código secuencial.
Lazy loading	Patrón de diseño que pospone la inicialización de un objeto hasta que es necesario.
MVVM	Model-View-ViewModel. Patrón arquitectónico que separa la lógica de negocio (Model) de la interfaz de usuario (View) mediante un intermediario (ViewModel).
Nominatim	Herramienta de geocodificación de código abierto basada en datos de OpenStreetMap.

Término	Definición
Offline-first	Estrategia de diseño en la que la aplicación está optimizada para funcionar sin conexión, sincronizando los datos cuando la conectividad está disponible.
OpenStreetMap	Proyecto colaborativo para crear un mapa libre y editable del mundo.
ORM	Object-Relational Mapping. Técnica de programación que permite convertir datos entre el sistema de tipos orientado a objetos y la base de datos relacional.
OSMDroid	Biblioteca Android para visualización de mapas basados en OpenStreetMap sin necesidad de servicios externos de pago.
Photon	Geocodificador de código abierto desarrollado por Komoot, basado en datos de OpenStreetMap, diseñado para ejecutarse localmente.
PostgreSQL	Sistema de gestión de bases de datos objeto-relacional de código abierto.
Repository Pattern	Patrón de diseño que abstrae la capa de acceso a datos, desacoplando la lógica de negocio de los detalles de persistencia.
Room	Biblioteca de persistencia de Android que proporciona una capa de abstracción sobre SQLite.
Sealed class/interface	Tipo de Kotlin que representa una jerarquía restringida de clases. Útil para modelar estados finitos.
Spring Boot	Framework de Java que simplifica la creación de aplicaciones Spring con configuración automática.
Spring Security	Framework de seguridad para aplicaciones Spring que proporciona autenticación, autorización y protección contra vulnerabilidades comunes.
StateFlow	Implementación de Flow de Kotlin que siempre tiene un valor actual y emite actualizaciones a sus colectores.
TSP	Traveling Salesman Problem (Problema del Viajante de Comercio). Problema de optimización combinatoria NP-duro que busca la ruta más corta que visita un conjunto de ciudades exactamente una vez.
UUID	Universally Unique Identifier. Identificador de 128 bits que garantiza unicidad sin necesidad de coordinación central.
WorkManager	API de Android para la ejecución de tareas en segundo plano con garantías de ejecución incluso si la app se cierra o el dispositivo se reinicia.
ZXing	Biblioteca de procesamiento de códigos de barras multiformato (Zebra Crossing).

10. ANEXOS

ANEXO A — ESTRUCTURA DEL REPOSITORIO GITLAB

El código fuente del proyecto está disponible en el repositorio:

<https://gitlab.com/jrc191/slior-project>

La estructura principal del repositorio es la siguiente:

- backend/ — Código fuente del servidor Spring Boot (Java 17).
- mobile-app/ — Código fuente de la aplicación Android (Kotlin 2.0.21).
- docs/ — Documentación técnica adicional.
- README.md — Instrucciones de instalación y despliegue.

ANEXO B — MANUAL DE INSTALACIÓN DEL BACKEND

Requisitos previos:

- Java 17 (OpenJDK 17 o superior).
- PostgreSQL 15 en ejecución local o remoto.
- Maven 3.8+ (o usar el wrapper incluido).

Pasos de instalación:

10. Clonar el repositorio: `git clone https://gitlab.com/jrc191/slior-project.git`
11. Crear la base de datos en PostgreSQL: `CREATE DATABASE slior;`
12. Configurar las variables de entorno en `application.properties`: URL de la BD, usuario, contraseña, secreto JWT.
13. Descargar el JAR de Photon (`photon-1.0.1.jar`) y los datos de OSM para España en el directorio `photon/`.
14. Compilar y ejecutar: `./mvnw spring-boot:run`
15. El servidor arrancará en `http://localhost:8080`. Photon se iniciará automáticamente en el puerto 2322.

ANEXO C — MANUAL DE INSTALACIÓN DE LA APLICACIÓN ANDROID

Requisitos previos:

- Dispositivo Android con versión 7.0 (API 24) o superior.
- Activar la instalación de fuentes desconocidas en los ajustes del dispositivo (para instalar la APK directamente).

Pasos de instalación:

16. Descargar el archivo APK desde la carpeta Programas/ del soporte de entrega.
17. Transferir el APK al dispositivo Android mediante USB o correo electrónico.
18. Abrir el archivo APK en el dispositivo y pulsar 'Instalar'.
19. Conceder los permisos solicitados: Ubicación (para la optimización de rutas) y Cámara (para el escaneo de códigos de barras).
20. Abrir la aplicación SLIOR e introducir las credenciales proporcionadas por el administrador.

Nota: La URL del servidor backend está configurada en tiempo de compilación en el archivo `build.gradle.kts`. Para apuntar a un servidor diferente, es necesario recompilar la aplicación.

ANEXO D — LICENCIA DEL PROYECTO

El proyecto SLIOR se distribuye bajo la licencia MIT (Massachusetts Institute of Technology License). Esta licencia es de tipo permisivo, lo que significa que permite el uso, copia, modificación, fusión, publicación, distribución, sublicencia y/o venta del software, con la única condición de mantener el aviso de copyright original.

Las bibliotecas de terceros utilizadas en el proyecto tienen las siguientes licencias:

Biblioteca	Licencia
Spring Boot / Spring Security / Spring Data JPA	Apache License 2.0
PostgreSQL JDBC Driver	BSD License
jjwt (JJWT)	Apache License 2.0
Jetpack Compose / Android Jetpack	Apache License 2.0
Room / Hilt / WorkManager	Apache License 2.0
Retrofit / OkHttp	Apache License 2.0
OSMDroid	Apache License 2.0
Photon Geocoder (Komoot)	Apache License 2.0
ZXing	Apache License 2.0
OpenStreetMap (datos cartográficos)	Open Database License (ODbL)
Space Grotesk (fuente)	SIL Open Font License 1.1

Todas las tecnologías utilizadas en SLIOR son compatibles con su distribución bajo la licencia MIT.

ANEXO E — ENDPOINTS DE LA API REST

Método	Endpoint	Auth	Descripción
POST	/auth/v1/register	No	Registra un nuevo usuario. Body: RegisterRequest.
POST	/auth/v1/login	No	Autentica un usuario. Body: LoginRequest. Response: AuthResponse con JWT.
GET	/health	No	Comprueba el estado del servidor. Response: {status: 'UP'}.
GET	/api/v1/geocode/search?q=	No	Busca direcciones. Param q: texto a geocodificar.
POST	/api/v1/routes	JWT	Crea una nueva ruta. Body: CreateRouteRequest.
GET	/api/v1/routes/{id}	JWT	Obtiene el detalle de una ruta por su ID.
GET	/api/v1/routes/repartidor/{id}	JWT	Obtiene todas las rutas de un repartidor.
DELETE	/api/v1/routes/{id}	JWT	Elimina (lógicamente) una ruta.
POST	/api/v1/routes/{id}/optimize	JWT	Optimiza el orden de las paradas de una ruta. Body: OptimizeRouteRequest.